metrics.py — technical

`src/moe_pruning/evaluation/metrics.py · 56 lines · moe_pruning.evaluation.metrics`

Purpose

Pure functions over per-example results.

Formulas

```
mean_token_loss = sum(loss_sums) / sum(token_counts); mean_example_loss = mean_i(loss_sum_i / count_i)

perplexity = exp(mean_token_loss) (causal LM only)

relative_increase = 100 * (loss - base) / base; agreement = matching argmax positions / scored positions
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`summarize_losses`

```
def summarize_losses(loss_sums: list[float], token_counts: list[int], perplexity: bool=False) -> dict
```

lines 16-28

Mean token loss, mean example loss, counts (and perplexity for causal LMs).

`compare_to_baseline`

```
def compare_to_baseline(stage: dict, baseline: dict) -> dict
```

lines 31-37

Absolute and relative loss change against the original model.

`per_example_comparison`

```
def per_example_comparison(stage_losses: torch.Tensor, base_losses: torch.Tensor) -> dict
```

lines 40-47

Mean / median / max per-example loss change and share of examples worse.

`prediction_agreement`

```
def prediction_agreement(stage_preds: list[list[int]], base_preds: list[list[int]]) -> float
```

lines 50-56

Fraction of target positions whose teacher-forced argmax matches the baseline.